طراحی و پیاده‌سازی سیستم بینایی ماشین برای مانیتورینگ صنعتی و تحلیل حضور افراد

1. خلاصه اجرایی

توسعه یک سیستم بینایی ماشین بلادرنگ و پایدار برای محیط‌های صنعتی، نیازمند معماری دقیقی است که بتواند بر چالش‌های بنیادین نظیر انسداد تجهیزات، تغییرات نوری، و پیچیدگی‌های استریم ویدئویی غلبه کند. هدف از این تحقیق، ارائه یک معماری جامع، انتخاب دقیق تکنولوژی‌ها، و تدوین یک نقشه راه عملیاتی برای ساخت کمینه محصول پذیرفتنی (MVP) با قابلیت ارتقا به محیط تولید (Production) است. با بررسی عمیق مقالات علمی منتشر شده از سال ۲۰۲۰ تا ۲۰۲۶ و ارزیابی بنچمارک‌های سخت‌افزاری، ترکیبی از معماری تشخیص شیء YOLO11، الگوریتم ردیابی ByteTrack، و پایگاه داده سری‌زمانی TimescaleDB به‌عنوان بهینه‌ترین مسیر توسعه انتخاب شده است. تحلیل‌ها نشان می‌دهد که برای محیط‌های صنعتی، انتقال از رهیافت‌های متکی بر فریم به رهیافت‌های متکی بر زمان مطلق (Wall-clock)، و همچنین استفاده از ماشین‌های حالت (State Machines) با قابلیت خنثی‌سازی نویز (Debouncing)، دقت محاسبه زمان توقف (Dwell Time) را به شکل چشمگیری افزایش می‌دهد. این گزارش تمامی ابعاد مهندسی اعم از مدیریت هویت، هندسه نواحی، معماری همزمانی (Concurrency)، و سناریوهای آزمون را با جزئیات کامل و مبتنی بر شواهد علمی تبیین می‌نماید.

2. تعریف دقیق مسئله

در یک محیط کارخانه‌ای، ماشین‌آلات متعددی وجود دارند که حریم پیرامونی آن‌ها بر روی کف سالن با خطوط زرد رنگ مشخص شده است و این نواحی تحت عنوان Zone شناخته می‌شوند. دوربین‌های مداربسته تحت شبکه (CCTV/IP Cameras) به صورت پیوسته بر این نواحی نظارت دارند. هدف نهایی، توسعه سیستمی است که بتواند با استفاده از یادگیری عمیق، پردازش بلادرنگ جریان‌های ویدئویی را انجام داده و حضور افراد را در فریم‌ها تشخیص دهد. سیستم باید قادر باشد افراد را در طول زمان با یک شناسه یکتا (Track ID) ردیابی کرده و مختصات مکانی آن‌ها را با هندسه نواحی تعریف‌شده تطبیق دهد.

مسئله اصلی در این سناریو، ثبت دقیق زمان ورود (Entry) و خروج (Exit) و در نهایت محاسبه مدت زمان حضور (Dwell Time) هر فرد در هر Zone است. سیستم باید در برابر انسدادهای کوتاه‌مدت (مانند عبور فرد از پشت یک دستگاه) کاملاً مقاوم بوده و از ایجاد نشست‌های (Sessions) منقطع جلوگیری کند. در نهایت، تمامی این رویدادها باید در یک پایگاه داده ذخیره شوند تا امکان گزارش‌گیری‌های تحلیلی بر اساس شخص، ماشین، ناحیه، و زمان فراهم گردد. نمونه خروجی مورد انتظار شامل شناسه فرد، نام دستگاه، زمان دقیق ورود و خروج، و مدت زمان حضور بر حسب ثانیه است.

3. نیازمندی‌های عملیاتی

سیستم مانیتورینگ صنعتی باید مجموعه‌ای از نیازمندی‌های عملیاتی را به صورت بی‌نقص برآورده سازد. نخستین نیاز، توانایی اتصال پایدار به جریان‌های RTSP دوربین‌ها و مدیریت خودکار قطعی‌ها است. در لایه پردازش بینایی، سیستم باید هر فرد حاضر در میدان دید را با استفاده از مدل‌های تشخیص شیء شناسایی کرده و یک جعبه محصورکننده (Bounding Box) به همراه سطح اطمینان (Confidence Score) تولید کند. پس از آن، ماژول ردیابی باید به صورت پیوسته شناسه‌های یکتایی را به این جعبه‌ها اختصاص دهد.

در لایه منطق مکانی، سیستم نیازمند ارزیابی قرارگیری مختصات فیزیکی افراد در داخل چندضلعی‌های (Polygons) تعریف‌شده است. این ارزیابی باید به یک ماشین حالت (State Machine) متصل شود که رویدادهای ورود و خروج را مدیریت کند. ماشین حالت موظف است با اعمال مکانیزم‌های تأخیر و فیلترسازی نوسانات مرزی، از تولید رویدادهای کاذب جلوگیری نماید. در نهایت، داده‌های تولید شده شامل نشست‌های زمانی باید در یک ساختار رابطه‌ای و متناسب با داده‌های سری‌زمانی ذخیره شده و از طریق یک داشبورد تعاملی به کاربران نهایی نمایش داده شوند.

4. نیازمندی‌های غیرعملیاتی

برای اطمینان از کارایی سیستم در محیط واقعی، نیازمندی‌های غیرعملیاتی سخت‌گیرانه‌ای باید لحاظ شوند. سیستم باید کاملاً بلادرنگ (Real-Time) عمل کند، بدین معنا که قادر به پردازش حداقل ۱۵ الی ۳۰ فریم بر ثانیه به ازای هر جریان ویدئویی باشد.

مقیاس‌پذیری نرم‌افزار باید به گونه‌ای طراحی شود که ارتقای سیستم از یک دوربین در فاز MVP به چندین دوربین در فاز نهایی، تنها با افزودن منابع سخت‌افزاری و بدون نیاز به بازنویسی معماری پایه امکان‌پذیر باشد.
دقت بالا در ثبت رویدادها حیاتی است؛ نرخ ورودها و خروج‌های کاذب (False Entry/Exit) باید به حداقل ممکن برسد. پایداری سیستم در برابر نوسانات شبکه و اختلالات دوربین‌ها نیز الزامی است. فرآیندهای نرم‌افزاری باید به صورت خودکار توانایی بازیابی (Auto-recovery) را داشته باشند و در صورت افت فریم یا پردازش دیرهنگام، خطایی در محاسبه زمان توقف (Dwell Time) ایجاد نگردد. امنیت و حریم خصوصی داده‌ها، به ویژه در ذخیره‌سازی اطلاعات مرتبط با کارگران، از دیگر الزامات اساسی این معماری به شمار می‌رود.

5. مرور ادبیات علمی (Literature Review)

تحقیقات سال‌های اخیر در حوزه‌های تشخیص اشیاء، ردیابی چندگانه، و تحلیل ویدئویی، مسیر روشنی را برای انتخاب الگوریتم‌های بهینه در محیط‌های صنعتی ترسیم کرده‌اند. تکامل معماری‌های تشخیص از مدل‌های مبتنی بر لنگر (Anchor-based) به رویکردهای بدون نیاز به سرکوب حداکثر غیرمجاز (NMS-free) مانند YOLOv10 و RT-DETR، تأخیر استنتاج را به شدت کاهش داده است[1]. در حوزه ردیابی، عبور از مدل‌های صرفاً متکی بر ویژگی‌های ظاهری (مانند DeepSORT) به مدل‌های متکی بر تطبیق همه‌جانبه مشاهدات (مانند ByteTrack و OC-SORT)، مقاومت ردیاب‌ها را در برابر انسدادهای فیزیکی افزایش داده است[3].
جدول زیر خلاصه‌ای از مقالات کلیدی و تأثیرگذار در این معماری را نشان می‌دهد:

عنوان مقاله	نویسندگان / سال	روش کلیدی و معیارها	ارتباط با پروژه MVP
YOLOv10: Real-Time End-to-End Object Detection	Wang et al., 2024	حذف NMS با تخصیص دوگانه، کاهش تأخیر در T4 تا 1.8x.	اثبات کارایی مدل‌های بدون NMS برای پردازش‌های بلادرنگ صنعتی با تأخیر بسیار پایین[1].
ByteTrack: Multi-Object Tracking by Associating Every Detection Box	Zhang et al., 2022	تطبیق جعبه‌های با اطمینان پایین در فاز دوم. MOTA: 80.3 در MOT17.	بهترین گزینه برای حفظ هویت افراد در حین انسداد بدون نیاز به محاسبات سنگین Re-ID[3].
BoT-SORT: Robust Associations Multi-Pedestrian Tracking	Aharon et al., 2022	جبران حرکت دوربین و بهبود فیلتر کالمن. HOTA: 65.0 در MOT17.	ارائه بالاترین دقت در ردیابی، اما هزینه محاسباتی آن استفاده از آن را برای MVP محدود می‌کند[8].
Observation-Centric (SORT (OC-SORT	Cao et al., 2023	اصلاح نویزهای فیلتر کالمن با تمرکز بر مشاهدات فیزیکی.	راهکاری مقاوم برای حرکات غیرخطی، مناسب برای دوربین‌های متحرک یا زاویه‌دار[5].
Customer Context	Xu et al., 2025	ترکیب تشخیص موقعیت و	تأکید بر اهمیت استفاده از

Analysis and Dwell-Time Estimation		استخراج معنایی زمان توقف در نواحی مسقف.	زمان مطلق به جای شمارش فریم برای محاسبه دقیق Dwell Time.[11]
TimescaleDB vs PostgreSQL Benchmark	Kiefer, 2017 ((Updated Data	معماری Hypertable برای داده‌های سری‌زمانی. 20x سرعت درج بالاتر.	اثبات برتری TimescaleDB برای مدیریت رویدادهای عظیم تولید شده توسط سیستم‌های مانیتورینگ ویدئویی[12].

این شواهد علمی نشان می‌دهند که برای دستیابی به عملکرد بلادرنگ، باید از مدل‌های سبک با تکنیک‌های تطبیق هندسی پیشرفته استفاده نمود تا سربار پردازشی شبکه‌های عصبی عمیق استخراج‌کننده ویژگی ظاهری (Appearance Feature Extractors) حذف گردد.

6. مقایسه مدل‌های تشخیص افراد

انتخاب معماری تشخیص شیء، تأثیر مستقیمی بر دقت سیستم و هزینه سخت‌افزاری دارد. خانواده YOLO به دلیل تعادل بی‌نظیر میان سرعت و دقت، همواره انتخاب اول در سیستم‌های بلادرنگ بوده است. با این حال، تفاوت‌های ساختاری مهمی میان نسخه‌های اخیر وجود دارد. مدل RT-DETR به‌عنوان یک معماری مبتنی بر ترانسفورمر، با رمزگذار هیبریدی خود استخراج ویژگی‌های متنی را بهبود بخشیده و نیاز به پردازش پسین (NMS) را از بین برده است[2]. با این وجود، پیاده‌سازی RT-DETR در دقت‌های عددی پایین (مانند INT8) مستعد افت دقت قابل توجهی است و روی کارت‌های میان‌رده، سرعت پایین‌تری نسبت به معماری‌های CNN-based دارد[15].

مدل YOLOv10 با معرفی تخصیص دوگانه (Dual Label Assignments)، رویکرد NMS-free را به دنیای CNNها آورد و توانست تأخیر را تا ۴۶ درصد نسبت به نسخه‌های قبلی کاهش دهد[1]. اما جدیدترین تحول، مدل YOLO11 است که با جایگزینی بلوک‌های سنتی با ماژول‌های C3K2 و C2PSA، توانایی شبکه را در حفظ ویژگی‌های فضایی اشیاء کوچک و ساختارهای باریک در محیط‌های شلوغ صنعتی افزایش داده است[17]. بررسی‌ها نشان می‌دهد که نسخه YOLO11m در دقت FP16 با استفاده از TensorRT می‌تواند سرعت خیره‌کننده‌ای (بالای ۱۵۰ فریم بر ثانیه) روی سخت‌افزارهای مدرن ارائه دهد[18].

مدل	معماری پایه	mAP (COCO)	مزایا برای محیط صنعتی	معایب / محدودیت‌ها
YOLOv8	CNN ((Anchor-free	بسیار بالا	پایدارترین اکوسیستم استقرار و جامعه کاربری عظیم[19].	نیاز به NMS که باعث افزایش تأخیر پردازشی می‌شود.
YOLOv10	CNN ((NMS-free	رقابتی	کاهش شدید تأخیر به دلیل حذف NMS[1].	در مواجهه با اشیاء بسیار کوچک، دقت کمتری نسبت به نسخه‌های بعدی دارد.
YOLO11	CNN +	برترین	عملکرد عالی در محیط‌های با موانع و	استفاده از ماژول‌های توجه کمی به

	Attention		پس‌زمینه‌های شلوغ[2].	پیچیدگی حافظه می‌افزاید.
RT-DETR	Transformer	رقابتی	ساختار End-to-End و دقت بالا در جزئیات متنی[2].	افت دقت شدید در فرآیند Quantization به INT8[15].

تصمیم برای MVP:
توصیه می‌شود از مدل **YOLO11s (Small)** یا **YOLO11m (Medium)** استفاده شود. برای فاز MVP، استفاده از وزن‌های از پیش‌آموزش‌دیده (Pre-trained) بر روی دیتاست COCO برای کلاس person کفایت می‌کند. در صورت وجود چالش‌هایی نظیر نور کم، تارشدگی ناشی از حرکت (Motion Blur) و پوشش‌های کاملاً یکپارچه صنعتی، مدل باید با استراتژی زیر Fine-tune شود:

- **حجم دیتاست:** ۱۰۰۰ الی ۲۰۰۰ تصویر از زاویه دوربین‌های کارخانه.
- **رزولوشن تصویر:** ۶۴۰ در ۶۴۰ پیکسل.
- **تقسیم داده‌ها:** ۸۰٪ آموزش، ۱۰٪ اعتبارسنجی، ۱۰٪ آزمون.
- **Augmentation:** استفاده از موزاییک (Mosaic) و تغییرات گامای نوری برای شبیه‌سازی شرایط محیطی[21].
- **خروجی:** مدل نهایی با استفاده از TensorRT در دقت FP16 صادر (Export) خواهد شد تا بیشترین نرخ فریم (FPS) حاصل گردد.

7. مقایسه الگوریتم‌های ردیابی

الگوریتم ردیابی چندگانه اشیاء (MOT) وظیفه دارد میان جعبه‌های تشخیص داده شده در فریم‌های متوالی، ارتباط منطقی و پیوسته برقرار کند. روش‌های کلاسیک مانند SORT صرفاً بر اساس موقعیت هندسی (IoU) و فیلتر کالمن عمل می‌کردند که در صورت بروز انسداد (Occlusion)، به سرعت دچار تغییر شناسه (ID Switch) می‌شدند. برای حل این مشکل، DeepSORT با معرفی استخراج ویژگی‌های ظاهری (Appearance Embedding) مقاومت ردیاب را بالا برد[22]. اما استخراج ویژگی ظاهری، به خصوص برای کارگرانی که لباس یکسان دارند، بسیار پرهزینه و گاهاً گمراه‌کننده است.
مدل‌های نوین رویکرد متفاوتی را اتخاذ کرده‌اند. الگوریتم ByteTrack با این ایده بنا شده است که فیلتر کردن و حذف تشخیص‌های با سطح اطمینان پایین (Low-confidence) منجر به قطع شدن مسیر ردیابی در حین انسداد می‌شود. بنابراین، این الگوریتم در یک فرآیند دو مرحله‌ای، ابتدا تشخیص‌های با اطمینان بالا را تطبیق داده و سپس برای مسیرهای باقی‌مانده، از تشخیص‌های با اطمینان پایین بهره می‌برد[3]. از سوی دیگر، BoT-SORT با جبران حرکت دوربین و اصلاح بردار حالت فیلتر کالمن، بالاترین دقت (MOTA/HOTA) را ارائه می‌دهد اما سرعت آن به دلیل پیچیدگی‌های محاسباتی، حدود ۳۰ درصد کمتر از ByteTrack است[8]. الگوریتم OC-SORT نیز با تمرکز بر مشاهدات فیزیکی به جای پیش‌بینی‌های صرفاً خطی کالمن، برای حرکات غیرخطی بسیار کارآمد است[5].

الگوریتم	رویکرد تطبیق (Association)	جبران انسداد (Occlusion)	نیاز به پردازش Re-ID	سرعت استنتاج
DeepSORT	IoU + بردارهای ویژگی ظاهری	حفظ Track بر اساس شباهت ظاهری	بله (مدل CNN مجزا)	متوسط[22]

ByteTrack	تطبیق دوگانه (High/Low) (Confidence	استفاده از تشخیص‌های مبهم (Low-score)	خیر	بسیار بالا[3]
BoT-SORT	IoU + حرکت دوربین + Re-ID (اختیاری)	پیش‌بینی دقیق‌تر با کالمن اصلاح‌شده	انتخابی	متوسط رو به پایین[9]
OC-SORT	مشاهدات مستقیم (Observation-c) (entric	اصلاح مسیر پس از پایان انسداد	خیر	بالا[5]

تصمیم برای MVP: الگوریتم **ByteTrack** به عنوان گزینه نهایی برای MVP انتخاب می‌شود. دلایل این انتخاب شامل معماری ساده و بدون نیاز به استخراج ویژگی ظاهری، سرعت فوق‌العاده بالا که امکان اجرای همزمان روی چندین دوربین را فراهم می‌کند، و عملکرد بی‌نظیر در حفظ شناسه افراد هنگام قرارگیری در پشت دستگاه‌های صنعتی (جایی که نمره تشخیص از ۰.۹ به ۰.۲ افت می‌کند اما جعبه حذف نمی‌شود) است[23].

8. مدیریت هویت (Track ID در مقابل Person ID)

در سیستم‌های تحلیل ویدئویی، تفاوت بنیادینی میان Track ID (شناسه موقت تخصیص‌یافته توسط ردیاب در یک میدان دید پیوسته) و Person ID (شناسه سراسری بیومتریک یک فرد) وجود دارد.

معماری A (تنها مبتنی بر ردیابی)، یک Track ID به فرد اختصاص می‌دهد تا زمانی که او در کادر دوربین است. اگر فرد پشت یک ستون برود و بافر ردیاب (Track Buffer) به پایان برسد، با ظهور مجدد فرد، یک Track ID جدید تولید می‌شود. معماری B (ردیابی + شناسایی مجدد ظاهری یا Re-ID)، از مدل‌های عمیق برای استخراج بردار ویژگی (Embedding) از ظاهر فرد استفاده می‌کند تا حتی در صورت خروج از کادر و ورود مجدد، شناسه سراسری (Person ID) او حفظ شود[25].

تحلیل برای MVP: پیاده‌سازی معماری B (Re-ID) به دلیل تغییرات ظاهری شدید در زوایای مختلف دوربین، شباهت لباس کارگران، و نیاز به مدیریت یک پایگاه داده برداری (Vector DB) برای جستجوی ویژگی‌ها، پیچیدگی سیستم را چندین برابر می‌کند[25]. برای محاسبه Dwell Time در یک Zone پیرامون ماشین‌آلات، نیازی به دانستن هویت واقعی فرد نیست؛ صرفاً تعیین اینکه "یک موجودیت فیزیکی پیوسته" چه مدت در منطقه حضور داشته است، کفایت می‌کند. از این رو، **Architecture A (Tracking-only)** برای MVP انتخاب می‌شود تا تمرکز بر روی پایداری سیستم در یک دوربین معطوف گردد.

9. بازنمایی ناحیه و موقعیت (Zone Detection)

برای بررسی حضور یک فرد در ناحیه‌ای مشخص از کف کارخانه، باید جعبه محصورکننده (Bounding Box) فرد را با چندضلعی (Polygon) مربوط به ناحیه مقایسه کرد. روش‌های متفاوتی برای این منظور وجود دارد:

1. **مرکز جعبه (Bounding Box Center):** در دوربین‌های نصب شده در ارتفاع زاویه‌دار، مرکز جعبه معمولاً در ناحیه بالاتنه فرد قرار می‌گیرد که ممکن است خارج از ناحیه کف باشد.
2. **پایین-مرکز (Foot Point):** مختصات (x_center, y_max). این نقطه دقیقاً محل تماس پای فرد با زمین را تقریب می‌زند.
3. **اشتراک مساحت (Box Intersection):** محاسبه مساحت مشترک جعبه و چندضلعی که نیازمند محاسبات هندسی سنگین‌تری است.

تصمیم برای MVP: استفاده از نقطه تماس پا یا **Foot Point**. این روش از نظر محاسباتی بسیار سبک بوده و بیشترین همخوانی را با مکان فیزیکی فرد روی زمین در تصاویر دوربین‌های مداربسته دارد. برای ارزیابی داخل بودن این نقطه در چندضلعی دستی تعریف‌شده، از الگوریتم ری‌کستینگ (Ray-Casting) موجود در توابع استانداردی نظیر cv2.pointPolygonTest استفاده

خواهد شد[28].

10. تشخیص ورود و خروج (Entry/Exit Detection)

حضور کارگر در نزدیکی مرزهای یک ناحیه و خطاهای ریز الگوریتم ردیابی می‌تواند باعث نوسان نقطه Foot Point در مرز چندضلعی شود. این امر پدیده‌ای مخرب به نام Jittering ایجاد می‌کند که در صورت عدم کنترل، ده‌ها رویداد ورود و خروج کاذب در کسری از ثانیه تولید می‌نماید. برای جلوگیری از این مشکل، استفاده از یک ماشین حالت (State Machine) با مکانیزم تأخیر (Debouncing) الزامی است.

معماری ماشین حالت شامل وضعیت‌های OUTSIDE و INSIDE است.

- زمانی که نقطه حضور فرد در وضعیت OUTSIDE قرار دارد، سیستم فریم‌های داخل بودن را می‌شمارد. اگر فرد برای حداقل N فریم متوالی در داخل ناحیه شناسایی شود، وضعیت به INSIDE تغییر کرده و رویداد ZONE_ENTER با برچسب زمانیِ نخستین فریمِ شناسایی، ثبت می‌شود.
- هنگامی که وضعیت INSIDE است، خروج از ناحیه برای حداقل M فریم متوالی ارزیابی می‌شود تا رویداد ZONE_EXIT قطعی گردد.

توصیه پارامترها: برای یک جریان ویدئویی با نرخ 15 فریم بر ثانیه، مقادیر $N = 10$ (تقریباً ۰.۶ ثانیه) و $M = 15$ (حدود ۱ ثانیه) برای فیلتر کردن نوسانات مرزی و حفظ رویدادهای معتبر پیشنهاد می‌گردد.

11. محاسبه زمان توقف (Dwell Time)

محاسبه دقیق مدت زمان حضور (Dwell Time)، هسته اصلی ارزش‌گذاری این سیستم است. اتکا به فرمول $Frame_Count / FPS$ در محیط‌های عملیاتی کاملاً اشتباه است، زیرا پرش‌های جریان RTSP، افت فریم در لایه شبکه، و تاخیرهای پردازشی باعث تغییر نرخ فریم واقعی می‌شوند[11].

تصمیم مهندسی:

زمان توقف باید مطلقاً مبتنی بر زمان ساعت سرور (Wall-clock Timestamp) محاسبه شود. در لحظه‌ای که فریم ویدئویی از جریان RTSP دریافت می‌شود، یک برچسب زمانی دقیق (مثلاً توسط time.time() در پایتون) به شیء فریم الحاق می‌گردد. در ماشین حالت، Entry Timestamp و Exit Timestamp بر اساس این زمان‌های مطلق ذخیره شده و Dwell Time از تفاضل آن‌ها استخراج می‌شود. این روش، تمامی خطاهای ناشی از متغیر بودن FPS یا سربار محاسباتی را خنثی می‌سازد.

12. مدیریت انسداد و گم‌شدن موقت (Occlusion Handling)

یکی از چالش‌برانگیزترین سناریوها، پنهان شدن کارگر در پشت یک دستگاه صنعتی برای چند ثانیه است. در این حالت، الگوریتم تشخیص قادر به یافتن فرد نیست و ردیاب نیز شناسه او را در حالت مفقود (Lost) قرار می‌دهد. سیستم نباید با گم شدن فرد، فوراً وضعیت خروج (EXIT) را ثبت کند.

چرخه حیات یک مسیر (Track Lifecycle) باید به صورت زیر مدیریت شود:

1. **ACTIVE:** فرد در فریم تشخیص داده شده و مکان او به‌روز می‌شود.
2. **OCCLUDED/LOST:** فرد تشخیص داده نمی‌شود. ردیاب (ByteTrack) شناسه فرد را در یک حافظه پنهان (Track Buffer) نگه می‌دارد. اگر فرد در منطقه INSIDE مفقود شده است، ماشین حالت باید نشست را باز (OPEN) نگه دارد.
3. **REMOVED:** تعداد فریم‌های مفقود ماندن فرد از آستانه track_buffer_size فراتر می‌رود. ردیاب شناسه را حذف می‌کند. در این لحظه، ماشین حالت باید نشست را ببندد و رویداد ZONE_EXIT را با زمان آخرین لحظه مشاهده (Last Seen Timestamp) ثبت کند.

برای سیستمی که با 15 فریم بر ثانیه کار می‌کند، تنظیم track_buffer_size روی عدد 45 الی 60 (معادل ۳ تا ۴ ثانیه تحمل انسداد) بهینه‌ترین حالت برای جلوگیری از قطع شدن نشست‌های حضور است[31].

13. دریافت استریم (RTSP Architecture)

دریافت جریان‌های ویدئویی از دوربین‌های مداربسته از طریق پروتکل RTSP انجام می‌پذیرد. استفاده سنتی از تابع cv2.VideoCapture در OpenCV در محیط‌های پروداکشن به دلیل مسدود شدن (Blocking) فرآیند در زمان افت کیفیت شبکه، منجر به خرابی کل سیستم می‌شود.
تحلیل‌ها نشان می‌دهند که برای سیستم‌های صنعتی مقاوم، باید از رویکردهای پردازش ناهمگام و الگوهای تولیدکننده/مصرف‌کننده (Producer/Consumer) استفاده کرد. فریم‌ها توسط ابزارهایی نظیر PyAV یا پیوندهای بومی GStreamer در یک رشته مجزا (Thread) دریافت شده و در یک صف با اندازه محدود (مانند queue.Queue(maxsize=10)) قرار می‌گیرند[33]. در صورت تأخیر در پردازش هوش مصنوعی، فریم‌های قدیمی از صف دور ریخته می‌شوند (Frame Drop) تا سیستم همواره بر روی فریم‌های زنده و بلادرنگ عمل کند. مکانیزم اتصال مجدد (Reconnection) با تاخیر نمایی (Exponential Backoff) باید برای مدیریت قطعی‌های موقت شبکه در این ماژول تعبیه شود.

14. معماری همزمانی و بلادرنگ (Real-Time Architecture)

در زبان پایتون، وجود قفل مفسر سراسری (GIL) مانع از اجرای موازی واقعی در معماری‌های مبتنی بر Threading برای پردازش‌های سنگین پردازنده مرکزی (CPU-Bound) می‌شود. از این رو، یک معماری خطی (Architecture A) باعث گلوگاه پردازشی شدیدی خواهد شد.
برای MVP و قابلیت مدیریت تا ۴ دوربین همزمان، **Architecture B (مبتنی بر Multiprocessing)** انتخاب می‌شود:

- **پروسه ۱ (RTSP Capture):** به ازای هر دوربین یک پروسه سبک مسئول دریافت فریم‌ها و انتقال آن‌ها از طریق صف‌های مشترک (Shared Queues) است.
- **پروسه ۲ (AI Inference):** یک پروسه متمرکز که دسته‌های فریم‌ها (Batches) را دریافت کرده و با استفاده از مدل YOLO (بر روی GPU) و الگوریتم ردیابی، مختصات افراد را تولید می‌کند.
- **پروسه ۳ (Event & Business Logic):** پردازش هندسی (محاسبه حضور در نواحی)، اجرای منطق ماشین حالت و تولید رویدادها را بر عهده دارد و داده‌ها را به صورت ناهمگام (AsyncIO) به پایگاه داده ارسال می‌کند.

این تفکیک پروسه‌ها، بهره‌وری سخت‌افزار را به حداکثر رسانده و از مسدود شدن لایه هوش مصنوعی به دلیل تأخیرهای مربوط به پایگاه داده جلوگیری می‌کند.

15. پایگاه داده (Database Design)

داده‌های تولید شده توسط این سیستم، ماهیتی کاملاً سری‌زمانی (Time-series) دارند. پایگاه داده باید قابلیت درج (Insert) سریع رویدادها را بدون قفل کردن جداول داشته باشد. بر اساس بنچمارک‌های موجود، پایگاه داده **PostgreSQL** در ترکیب با افزونه **TimescaleDB** می‌تواند سرعت درج را در مقایسه با PostgreSQL خام تا ۲۰ برابر افزایش داده و امکان پرس‌وجوهای پیچیده تحلیلی را فراهم سازد[12].
ساختار پایگاه داده (SQL Schema) برای MVP به شرح زیر خواهد بود:

```SQL
CREATE EXTENSION IF NOT EXISTS timescaledb;

CREATE TABLE cameras (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    rtsp_url TEXT
);
```

```
CREATE TABLE
PRIMARY
VARCHAR 100
)

CREATE TABLE
PRIMARY
INT REFERENCES
INT REFERENCES
VARCHAR 100
-- ذخیره هندسه به صورت JSON
)

CREATE TABLE
PRIMARY  DEFAULT
INT REFERENCES
INT REFERENCES
INT REFERENCES
INT
NOT NULL
NUMERIC 10  2
VARCHAR 20  DEFAULT 'OPEN
)

-- پیکربندی Hypertable در TimescaleDB برای مدیریت داده‌های عظیم زمانی
'SELECT  'zone_sessions'  'entry_time
```

استفاده از Hypertableها باعث پارتیشن‌بندی خودکار داده‌ها بر اساس زمان شده و کارایی درج رویدادها را به شدت ارتقا می‌دهد[13].

16. موتور رویدادها (Event Engine)

سیستم نیازمند یک معماری مبتنی بر رویداد (Event-driven Architecture) برای جداسازی منطق تشخیص از ذخیره‌سازی است. در MVP، این موتور به کمک الگوی Observer پیاده‌سازی می‌شود. هنگامی که ماشین حالت ورود فردی را تأیید می‌کند، رویداد ZONE_ENTER منتشر شده و وضعیت رکورد در دیتابیس به صورت OPEN ایجاد می‌گردد. با صدور رویداد ZONE_EXIT، رکورد مربوطه پیدا شده، زمان خروج و duration_seconds محاسبه و به‌روزرسانی شده، و وضعیت به CLOSED تغییر می‌یابد.

17. ارزیابی (Evaluation)

برای تضمین صحت عملکرد سیستم در شرایط واقعی کارخانه، ارزیابی در سه لایه مستقل انجام می‌گیرد:

1. **لایه تشخیص (Detection):** معیار mAP50 و mAP50-95 برای سنجش دقت و فراخوانی الگوریتم YOLO11 در محیط‌های صنعتی و نورهای متغیر استفاده می‌شود.
2. **لایه ردیابی (Tracking):** معیارهای پیشرفته مانند HOTA (دقت ردیابی مرتبه بالا) و IDF1 برای ارزیابی توانایی

ByteTrack در حفظ هویت افراد به خصوص در حین تقاطع مسیره و انسداد استفاده می‌شوند[35].

3. **لایه منطق کسب‌وکار (Business Logic):** حیاتی‌ترین بخش ارزیابی است. معیارهای MAE (خطای مطلق میانگین) و RMSE برای سنجش میزان دقت در محاسبه Dwell Time به کار می‌روند. همچنین نرخ ورودها و خروج‌های اشتباه (False Entry/Exit) برای تنظیم پارامترهای N و M در ماشین حالت به کار گرفته خواهد شد.

18. سناریوهای آزمون (Test Scenarios)

برای اعتبارسنجی پایداری سیستم، مجموعه‌ای از ۲۰ سناریوی واقعی طراحی شده است که برخی از کلیدی‌ترین آن‌ها در جدول زیر ارائه گردیده است:

شناسه	سناریو (Input)	خروجی مورد انتظار (Expected Output)	شرط شکست (Failure Condition)
۱	ورود تک‌نفره واضح	ثبت یک رویداد ENTRY در لحظه ورود	ثبت ورودهای مکرر یا تاخیر طولانی.
۲	خروج تک‌نفره واضح	ثبت EXIT و محاسبه دقیق Dwell Time	باز ماندن وضعیت SESSION.
۳	تقاطع دو فرد (Crossing)	حفظ Track ID برای هر دو فرد	تغییر شناسه (ID Switch) بین دو نفر.
۴	انسداد موقت دستگاه	عدم قطع Session تا پایان track_buffer	ثبت خروج در حین پنهان بودن فرد.
۵	نوسان روی مرز ناحیه	خنثی‌سازی نوسانات (بدون ثبت رویداد)	تولید صدها سطر ENTRY/EXIT کاذب.
۶	خروج از کادر دوربین	قطع ارتباط ردیاب و ثبت رویداد EXIT	باز ماندن ردیاب در حافظه بیش از حد مجاز.
۷	قطع ارتباط دوربین	توقف سیستم بدون اختلال در پروسه‌های دیگر	کرش کردن برنامه پایتون (Crash).
۸	تاریکی موقت (Low light)	تداوم ردیابی در صورت امکان یا درج در بافر	افت کامل سیستم و قطع نشست‌ها.

19. سخت‌افزار (Hardware)

انتخاب سخت‌افزار مناسب، ضامن اجرای بلادرنگ است. پردازش ویدئوی چندین دوربین نیازمند حافظه گرافیکی (VRAM) بالا و توان محاسباتی بهینه است. بررسی‌ها و بنچمارک‌های عملکرد کارت‌های گرافیک نشان می‌دهد که اجرای مدل YOLOv8/11x با

کیفیت FP16 بر بستر TensorRT روی کارت **NVIDIA RTX 3090 (24GB)** به سرعتی فراتر از ۸۰ فریم بر ثانیه برای تصاویر با رزولوشن بالا می‌رسد. در مورد مدل‌های کوچکتر مانند YOLO11s/m، این نرخ پردازش حتی می‌تواند به بیش از ۱۵۰ فریم بر ثانیه بالغ شود[18]. مزیت اصلی RTX 3090 حجم بالای VRAM آن (۲۴ گیگابایت) است که به معماری نرم‌افزاری اجازه می‌دهد از تکنیک بچینگ (Batch Inference) استفاده کند و پردازش ۴ تا ۸ دوربین همزمان 1080p را با هزینه‌ای منطقی و بسیار اقتصادی‌تر از سری کارت‌های RTX 4090 یا سری آداپتورهای سازمانی مدیریت نماید[18].

20. پیاده‌سازی پایتون (Python Architecture)

معماری نرم‌افزاری باید مقیاس‌پذیر و ماژولار باشد. ساختار درختی پروژه به شکل زیر تدوین می‌گردد:

```
/factory_monitoring
├── /configs
│   └── settings.yaml # پیکربندی RTSPها، مختصات Zoneها و پارامترهای N, M
├── /app
│   ├── /streams
│   │   └── rtsp_worker.py # پروسه‌های دریافت فریم از دوربین‌ها (PyAV)
│   ├── /detection
│   │   └── yolo_engine.py # بارگذاری موتور TensorRT برای YOLO11
│   ├── /tracking
│   │   └── bytetrack_engine.py# پیاده‌سازی منطق دو مرحله‌ای ByteTrack
│   ├── /logic
│   │   ├── geometry.py # توابع استخراج Foot Point و بررسی Polygon
│   │   └── state_machine.py # کلاس مدیریت وضعیت‌های ورود و خروج
│   └── /database
│       └── db_writer.py # پروسه ناهمگام ذخیره در TimescaleDB
├── /dashboard
│   └── app.py # اپلیکیشن بصری Streamlit
├── main.py # ارکستراتور اصلی (مدیریت پروسه‌های Multiprocessing)
└── docker-compose.yml # پیکربندی استقرار
```

21. مصورسازی و داشبورد (Dashboard)

داشبورد یکی از اجزای کلیدی MVP برای نمایش کارایی سیستم به ذی‌نفعان است. برای این منظور استفاده از پلتفرم **Streamlit** در ترکیب با بک‌اند پایگاه داده پیشنهاد می‌شود. در مقایسه با ترکیب FastAPI و React که نیاز به توسعه‌دهندگان مجرب فرانت‌اند دارد، Streamlit امکان خلق رابط‌های کاربری تحلیلی را مستقیماً با پایتون فراهم کرده و سرعت پیاده‌سازی MVP را تضمین می‌کند[37].

قابلیت‌های داشبورد شامل موارد زیر خواهد بود:

- **نظارت زنده:** نمایش تعداد افراد حاضر در هر ناحیه به صورت در لحظه.
- **تاریخچه نشست‌ها:** نمایش جدولی شامل Person ID (موقت)، دستگاه، ورود، خروج، و مدت زمان (ثانیه).
- **گزارش‌های تجمعی:** نمودارهای میله‌ای برای نشان دادن میزان اشغال ماشین‌آلات مختلف در بازه‌های زمانی دلخواه.

22. استقرار و مانیتورینگ (Deployment & Monitoring)

محیط‌های صنعتی نیازمند پایداری کامل نرم‌افزار هستند. بستر استقرار توصیه‌شده، سیستم عامل **Ubuntu Server** همراه با بستر کانتینری **Docker** و ابزار **NVIDIA Container Toolkit** است. این ابزارها امکان اختصاص مستقیم دسترسی GPU به کانتینرها را فراهم کرده و مشکلات ناشی از تداخل پیش‌نیازهای پایتون را کاملاً مرتفع می‌سازند. برای مانیتورینگ داخلی در فاز MVP، استفاده از ماژول استاندارد logging در پایتون با پیکربندی RotatingFileHandler کافی است تا اطلاعاتی نظیر افت

فریم‌ها، تاخیر در استنتاج، و قطعی دوربین‌ها در فایل ثبت گردد. انتقال این لاگ‌ها به سرویس‌های مانیتورینگ جامع‌تر (مانند Prometheus) به فازهای پروداکشن موکول می‌گردد. امنیت داده‌ها با استفاده از متغیرهای محیطی رمزنگاری‌شده در فایل .env و پیاده‌سازی دسترسی‌های شبکه محلی (Air-gapped) تأمین می‌شود[39].

23. تحلیل خطاها (Error Analysis)

چالش‌های احتمالی سیستم و راهکارهای مهندسی آن‌ها شامل موارد زیر است:

- **Detection Error:** عدم توانایی در شناسایی کارگر به دلیل نور ضعیف یا پوشش خاص. راهکار: Fine-tuning مدل روی داده‌های اختصاصی با اعمال Augmentation برای نور کم.
- **Tracking Error (ID Switch):** جابجایی هویت دو کارگر در زمان تقاطع مسیر. راهکار: تنظیم حساسیت بردارهای IoU در الگوریتم ByteTrack و افزایش پارامتر اطمینان.
- **Timestamp Error:** اختلاف زمان به دلیل بافر شدن استریم. راهکار: اتصال مطلق زمان کلاک سیستم (Wall-clock) به فریم در لحظه دریافت از شبکه[11].
- **Database Error:** تاخیر در درج به علت قفل شدن جداول. راهکار: استفاده از پردازش ناهمگام (Async) و مزایای Hypertable در TimescaleDB[13].

24. برنامه ۳۰ روزه توسعه MVP (30-Day MVP Plan)

برای ساخت و تحویل سیستم اولیه، یک برنامه چهار هفته‌ای فشرده و در عین حال واقع‌بینانه پیشنهاد می‌شود:

- **هفته اول (استریم و هوش مصنوعی):** راه‌اندازی محیط توسعه (Docker، TensorRT). پیاده‌سازی کلاس rtsp_worker.py مقاوم. اجرای مدل پایه YOLO11s روی جریان ویدئویی و سنجش FPS روی سخت‌افزار.
- **هفته دوم (ردیابی و هندسه):** ادغام ماژول ByteTrack با نتایج استنتاج YOLO. پیاده‌سازی ساختار چندضلعی‌ها (Zones) و نوشتن الگوریتم محاسبه موقعیت پای شخص (Foot Point) در ارتباط با مرزهای نواحی.
- **هفته سوم (منطق و داده):** توسعه ماشین حالت (State Machine) جهت فیلتر کردن نویزها و تولید رویدادهای ENTRY و EXIT قطعی. نصب پایگاه داده TimescaleDB و پیاده‌سازی پروسه‌های درج (Insert) ناهمگام.
- **هفته چهارم (یکپارچه‌سازی و داشبورد):** پیاده‌سازی رابط کاربری با Streamlit برای نمایش نتایج. انجام تست‌های گسترده روی ۲۰ سناریوی از پیش تعریف شده. بهینه‌سازی نهایی پروسه‌ها برای کنترل گلوگاه‌های CPU.

25. دامنه و ارتقا (MVP Scope vs Production Upgrade)

در نسخه MVP، تمرکز صرفاً بر روی ایجاد یک سامانه پایدار روی یک دوربین (Single-camera Pipeline) است که شامل تشخیص، ردیابی، ارزیابی نواحی دستی (Manual Polygons)، و محاسبه دقیق Dwell Time می‌شود. قابلیت‌های پیچیده‌ای نظیر تشخیص هویت سراسری با دوربین‌های متعدد (Person Re-ID) یا شناسایی خودکار خطوط زرد محیط، به دلیل تحمیل سربار پردازشی عظیم و عدم پایداری لازم در این فاز، حذف شده‌اند.

مسیر ارتقا به Production: در نسخه‌های آتی، پس از استقرار MVP و پایش عملکرد، گام‌های ارتقا شامل Fine-Tuning گسترده روی داده‌های اختصاصی کارخانه، مهاجرت به بسترهای توزیع‌شده نظیر Kubernetes، و افزودن ماژول استخراج‌گر ویژگی ظاهری برای ردیابی بین‌دوربینی (Multi-camera Identity) خواهد بود.

26. بلوپرینت نهایی و تصمیمات مهندسی (FINAL MVP BLUEPRINT)

- **Detector:** الگوریتم YOLO11s (ترکیب ایده‌آل دقت بالا و پردازش بسیار سریع در محیط‌های شلوغ).
- **Tracker:** الگوریتم ByteTrack (بهینه‌ترین ردیاب برای غلبه بر انسدادهای فیزیکی کوتاه‌مدت بدون نیاز به Re-ID پرهزینه).
- **Detection Model:** شروع با وزن‌های از پیش‌آموزش‌دیده COCO.
- **Zone Representation:** استفاده از چندضلعی دستی در مختصات دوبعدی تصویر (Manual 2D Polygon).
- **Point Representation:** نقطه پایین-مرکز جعبه یا Foot Point با مختصات (x_center, y_max).
- **Entry/Exit Algorithm:** استفاده از ماشین حالت مقاوم با پارامترهای تأخیری (مثلاً تأیید حضور پیوسته در ۱۰ فریم متوالی).

- **Occlusion Handling:** حفظ وضعیت OPEN برای نشست‌ها در بافر حافظه ByteTrack (تحمل تا ۳-۴ ثانیه مفقود ماندن کارگر).
- **Timestamp:** استفاده انحصاری از زمان کلاک سرور (Wall-clock) چسبانده شده به فریم در زمان دریافت اولیه.
- **RTSP / Concurrency:** بستر PyAV در معماری فرآیندهای چندگانه (Multiprocessing) با استفاده از صف‌های محدود ارتباطی (Shared Queues).
- **Database:** سیستم مدیریت رابطه‌ای PostgreSQL تقویت شده با افزونه TimescaleDB جهت درج عظیم داده‌های سری-زمانی.
- **Dashboard:** فریمورک Streamlit برای سرعت بی‌نظیر در ایجاد رابط تحلیلی داده‌ها.
- **GPU:** پردازنده گرافیکی NVIDIA RTX 3090 24GB.
- **Deployment:** استقرار بر روی Ubuntu Server از طریق Docker Compose.

نقطه شروع توسعه:

اگر قرار است استارت توسعه از فردا زده شود، اولین قدم کدنویسی مدل‌های هوش مصنوعی نیست؛ بلکه پیاده‌سازی یک ماژول ارتباط شبکه بی‌نقص برای دریافت جریان‌های RTSP است. یک برنامه پایتون بنویسید که جریان دوربین را باز کرده، فریم‌ها را در یک queue.Queue قرار دهد، مکانیزم پرش فریم (Frame Dropping) برای جلوگیری از تأخیر را فعال کند و توانایی بازیابی اتصال (Auto-reconnect) در زمان افت شبکه را داشته باشد. پایداری این ماژول، سنگ‌بنای اصلی تمام پردازش‌های هوش مصنوعی و محاسبات زمان توقف در لایه‌های بعدی خواهد بود.

Works cited

1. YOLOv10 NMS-Free Object Detection - Ultralytics YOLO, https://docs.ultralytics.com/models/yolov10
2. A Comparative Analysis of State-of-the-Art Object Detection Models, https://iotdigitaltwinplm.com/a-comparative-analysis-of-state-of-the-art-object-detection-models/
3. ByteTrack: Multi-object Tracking by Associating Every Detection Box, https://www.researchgate.net/publication/364624019_ByteTrack_Multi-object_Tracking_by_Associating_Every_Detection_Box
4. ByteTrack: Enhanced Multi-Object Tracking | PDF | Kalman Filter, https://www.scribd.com/document/623219311/ByteTrack
5. OCAFI: observation centric appearance free interpolation technique, https://www.tandfonline.com/doi/full/10.1080/23311916.2025.2532808
6. ultralytics/docs/en/models/yolov10.md at main - GitHub, https://github.com/ultralytics/ultralytics/blob/main/docs/en/models/yolov10.md
7. ByteTrack: Efficient Object Tracking - Emergent Mind, https://www.emergentmind.com/topics/bytetrack
8. BoT-SORT: Robust Associations Multi-Pedestrian Tracking, https://www.researchgate.net/publication/361630662_BoT-SORT_Robust_Associations_Multi-Pedestrian_Tracking
9. BoT-SORT: Robust Associations Multi-Pedestrian Tracking | alphaXiv, https://www.alphaxiv.org/abs/2206.14651
10. Top 7 Open Source Object Tracking Tools - Roboflow Blog, https://blog.roboflow.com/top-object-tracking-software/
11. Customer Context Analysis in Shopping Malls: A Method Combining, https://www.mdpi.com/1424-8220/25/3/649

12. TimescaleDB vs. Postgres for time-series: 20x higher inserts, 2000x, https://medium.com/timescale/timescaledb-vs-6a696248104e
13. Scaling Time-Series Data with TimescaleDB in PostgreSQL, https://blog.stackademic.com/scaling-time-series-data-with-timescaledb-in-postgresql-8cc3c5e8bb76
14. (PDF) Review of large YOLOv8 and RT-DETR energy efficiency on, https://www.researchgate.net/publication/403222343_Review_of_large_YOLOv8_and_RT-DETR_energy_efficiency_on_edge_devices_for_real-time_detection
15. RT-DETR (TAO 6.x) INT8 accuracy drops than FP16/FP32, https://forums.developer.nvidia.com/t/rt-detr-tao-6-x-int8-accuracy-drops-than-fp16-fp32/375782
16. Real-Time Fruit Detection and Video Analytics on Embedded Edge, https://arxiv.org/html/2609.13551v1
17. Enhanced obstacle detection for intelligent agricultural machinery, http://www.ijabe.net/article/doi/10.25165/j.ijabe.20261904.10644
18. Best GPU for YOLOv8 (FPS + Cost Efficiency) - GIGAGPU, https://gigagpu.com/best-gpu-for-yolov8/
19. LBA-YOLO: A novel lightweight approach for detecting micro-cracks, https://pmc.ncbi.nlm.nih.gov/articles/PMC12063840/
20. UAV Small Target Detection Method Based on Frequency ... - MDPI, https://www.mdpi.com/2504-446X/10/2/106
21. Enhanced obstacle detection for intelligent agricultural machinery, https://www.researchgate.net/publication/414082522_Enhanced_obstacle_detection_for_intelligent_agricultural_machinery_via_transfer_learning_and_improved_YOLO11
22. Multi-Object Tracking — DeepSORT, ByteTrack, OC-SORT, And The, https://www.forasoft.com/learn/ai-for-video-engineering/articles-ai/multi-object-tracking-deepsort-bytetrack-ocsort
23. An Introduction to BYTETrack: Multi-Object Tracking by Associating, https://datature.io/blog/introduction-to-bytetrack-multi-object-tracking-by-associating-every-detection-box
24. Small Objects in the Wild, Part 4: Tracking What You Can Barely See, https://savant-ai.io/blog/small-objects-tracking/
25. Person Re-Identification with Attribute-Guided, Robust-to-Low, https://www.mdpi.com/1424-8220/25/6/1819
26. Senior Computer Vision (Grade A) - Claude Skill - Skills Directory, https://www.skillsdirectory.com/skills/jantoniofc-senior-computer-vision
27. Virtual Try-on-Based Data Augmentation for Robust Person Re, https://www.mdpi.com/2571-6255/9/3/116
28. guides/parking-management/ · ultralytics · Discussion #11093 - GitHub, https://github.com/orgs/ultralytics/discussions/11093
29. YOLO Classification Model always returns l_classifier = None, https://forums.developer.nvidia.com/t/deepstream-7-yolo-classification-model-always-returns-l-classifier-none/346034
30. Modelling railway dwell time: A structured review and lifecycle,

https://research-repository.rmit.edu.au/ndownloader/files/61623010/1
31. Real-Time Multi-Object Tracking under Resource Constraints, https://repositum.tuwien.at/bitstream/20.500.12708/224661/1/Bernhart%20Costin%20-%202025%20-%20Real-Time%20Multi-Object%20Tracking%20under%20Resource...pdf
32. Small Objects in the Wild, Part 5: Blur, Sensors, and the Tiling Tax, https://savant-ai.io/blog/small-objects-deployment/
33. How can I implement Reconnection In the following pipeline, https://discourse.gstreamer.org/t/how-can-i-implement-reconnection-in-the-following-pipeline/4888
34. PostgreSQL + TimescaleDB - Tiger Data, https://www.tigerdata.com/blog/postgresql-timescaledb-1000x-faster-queries-90-data-compression-and-much-more
35. FastTracker: Real-Time and Accurate Visual Tracking - arXiv, https://arxiv.org/html/2508.14370v3
36. YOLOv8 Multi-Machine Benchmark: RTX 3090 vs Ryzen AI MAX+, https://www.reddit.com/r/ryzen/comments/1lzr7yq/yolov8_multimachine_benchmark_rtx_3090_vs_ryzen/
37. Why You Need FastAPI Between Snowflake and Streamlit - Medium, https://medium.com/@nazeer.td/the-power-trio-fastapi-snowflake-and-streamlit-1d211e78dc67
38. FastAPI and Streamlit: The Python Duo You Must Know About, https://towardsdatascience.com/fastapi-and-streamlit-the-python-duo-you-must-know-about-72825def1243/
39. Google Distributed Cloud air-gapped | Sovereign Cloud, https://cloud.google.com/distributed-cloud-air-gapped